

OPSsig Self-Evaluation Report (OPSsig-256, Reference Implementation)

1 Basic Algorithm Information

Item	Value
Algorithm category	Digital Signature
Algorithm name	OPSsig
Sub-function	Key generation, signature generation, and signature verification
Implementation version	Reference Implementation
Submission version label	x86 Reference Implementation
Parameter set / security level	OPSsig-256

2 Evaluation Environment

Item	Value
Processor model	Intel(R) Xeon(R) Platinum 8575C
Processor frequency (reported)	2800.000 MHz
Logical cores	4
Installed memory	15698744 KiB (14.97 GiB)
Architecture	x86_64
Operating system	Ubuntu 22.04.5 LTS
Kernel version	5.15.0-71-generic
Compiler version	gcc (Ubuntu 12.3.0-1ubuntu1~22.04.3) 12.3.0
Build tool version	GNU Make 4.3; cmake version 3.22.1
GNU time version	time (GNU Time) UNKNOWN
AVX2 support reported by host	yes
Evaluation start time (UTC)	2026-06-29T13:49:24Z
Evaluation end time (UTC)	2026-06-29T13:50:28Z

3 Functional Tests

The functional evaluation was executed through the current self-evaluation entry point `make selfeval`. The following checks passed:

- Known-answer test (KAT) generation completed successfully and wrote vectors to:
`Test_Vectors/reference/OPSsig-256/KAT_SIG_
OPSsig-256-reference.txt`
- Signature correctness regression test completed successfully according to the captured log.
- The correctness log reports the exported API sizes shown below and ends in a pass status.

Correctness log excerpt

```
CRYPTO_PUBLICKEYBYTES = 3392  
CRYPTO_SECRETKEYBYTES = 5056  
CRYPTO_BYTES = 5540  
OK
```

KAT generation log excerpt

```
Files have been saved in ../../../../Test_Vectors/reference/OPSsig-256.
```

4 Performance Tests

Performance measurements were collected for a fixed 64-byte message, in line with the current x86 self-evaluation guideline for digital signatures. Each core operation was sampled 1000 times. The table below reports average latency, average cycle count, and throughput.

Operation	Samples	Avg. ns	Avg. cycles	Ops/s
Key generation	1000	1293512.32	3621486.99	773.09
Sign (64 B)	1000	1672003.52	4681235.72	598.08
Verify (64 B)	1000	1216744.32	3406546.23	821.87

Supplemental Micro-Benchmarks

Operation	Avg. ns	Avg. cycles	Ops/s
polyvec_matrix_expand	977430.31	2736444.78	1023.09
poly_uniform_eta	28107.08	78541.63	35578.22
poly_uniform_gamma1	58124.32	162578.35	17204.50
poly_ntt	8076.46	22401.41	123816.67
poly_invntt_tomont	6747.01	18696.67	148213.78
poly_pointwise_montgomery	666.47	1719.00	1500442.63
poly_challenge	3894.97	10758.07	256741.06

5 Resource Consumption Tests

Static Memory Footprint

Static memory usage was estimated from the loaded binary sections of `test_static_mem_OPSSig` using the `size` tool. This load-only binary links the full implementation but intentionally executes no key generation, signing, or verification logic. The section totals capture code, read-only constants, initialized global data, and uninitialized global data, which corresponds to the guideline item “basic memory usage after loading but before executing algorithm operations”.

Metric	Value (Bytes)	Value (MiB)
Text segment (.text)	26963	0.03
Read-only data (.rodata)	2493	0.00
Initialized data (.data)	16	0.00
Uninitialized data (.bss)	232	0.00
Total static footprint	29704	0.03

Peak Runtime Memory

Runtime memory was measured over 100 iterations of key generation, signature generation, and verification using a fixed 64-byte message.

Metric	Value (Bytes)	Value (MiB)
Peak RSS from <code>getrusage</code>	2068480	1.97
VmPeak from <code>/proc/self/status</code>	2924544	2.79
VmHWM from <code>/proc/self/status</code>	1019904	0.97

6 Transmission and Storage Overheads

Artifact	Size (Bytes)	Source
Public key	3392	Exported API size
Secret key	5056	Exported API size
Signature buffer bound	5540	CRYPTO_BYTES
Signature test message	64	Fixed performance/memory input

7 Implementation Notes

Item	Value
Third-party dependencies	No external third-party cryptographic library is linked by the evaluated build. The evaluation uses the system compiler and standard build utilities already recorded above.
Randomness generation	Deterministic DRNG support from <code>drng.c</code> is used by the self-evaluation drivers; fixed seeds are used where reproducibility is required.
Compilation flags	<code>-std=c99 -Wpedantic -Wall -Wextra -O2</code>
Target ISA / portability note	Portable ISO C99 C code only; no ISA-specific intrinsics or assembly files are used in this version.
Optimization approach	Moderate compiler optimization with <code>-O2</code> under the ISO C99 reference-build profile.

8 Raw Evidence Index

Evidence item	Location / command
Implementation directory	Implementations/Reference_Implementation/ OPSsig-256
Primary evaluation command	make -C Implementations/Reference_ Implementation/OPSsig-256 selfeval
Self-evaluation orchestrator	Self_Evaluation/scripts/run_self_eval.sh
Environment collection script	Self_Evaluation/scripts/collect_env.sh
Static memory script	Self_Evaluation/scripts/measure_mem.sh
Metadata CSV	Self_Evaluation/results/reference/OPSsig-256/ latest/metadata.csv
Environment CSV	Self_Evaluation/results/reference/OPSsig-256/ latest/environment.csv
Correctness log	Self_Evaluation/results/reference/OPSsig-256/ latest/correctness.log
Performance CSV	Self_Evaluation/results/reference/OPSsig-256/ latest/speed.csv
Static memory CSV	Self_Evaluation/results/reference/OPSsig-256/ latest/memory_static.csv
Runtime memory CSV	Self_Evaluation/results/reference/OPSsig-256/ latest/memory_runtime.csv
KAT log	Self_Evaluation/results/reference/OPSsig-256/ latest/kat.log
Generated KAT vectors	Test_Vectors/reference/OPSsig-256/KAT_SIG_ OPSsig-256-reference.txt
Static-memory binary	Implementations/Reference_Implementation/ OPSsig-256/test_static_mem OPSsig
Input data note	Fixed 64-byte message for performance and memory tests; reproducible seeded DRNG for correctness and KAT generation.